

Banco de Dados Distribuído com Replicação Síncrona, Two-Phase Commit e Eleição de Líder

Edson Pimenta de Almeida [Universidade PUC MG | 1435541@sga.pucminas.br]

Felipe Carvalho de Paula Silva [Universidade PUC MG | 1319360@sga.pucminas.br]

João Lucas de Melo Quintão [Universidade PUC MG | 1165769@sga.pucminas.br]

Luís Augusto Starling Toledo [Universidade PUC MG | 1323060@sga.pucminas.br]

Juan Pablo Ramos de Oliveira [Universidade PUC MG | 1467614@sga.pucminas.br]

Luiz Gabriel Milione Assis [Universidade PUC MG | 1493412@sga.pucminas.br]

Túlio Gomes Braga [Universidade PUC MG | 1441272@sga.pucminas.br]

Resumo. Este trabalho apresenta um banco de dados chave-valor distribuído desenvolvido em Python com gRPC. As escritas são coordenadas por um líder eleito dinamicamente pelo algoritmo do valentão e aplicadas de forma atômica em todas as réplicas vivas por meio do protocolo *Two-Phase Commit* (2PC), com exclusão mútua distribuída por bloqueio de chaves, quórum majoritário e relógios lógicos de Lamport. O sistema tolera falhas em várias camadas: detecção por *heartbeat*, reeleição automática de líder, registro de escrita antecipada (WAL) para recuperação após quedas, transferência de estado para nós que reingressam e um protocolo de terminação que resolve transações pendentes quando o coordenador falha. A avaliação combina testes automatizados, injeção de falhas e um teste de estresse com clientes concorrentes, verificando ao final a consistência entre os nós.

Palavras-chave: Banco de Dados Distribuído, gRPC, Two-Phase Commit, Eleição de Líder, Tolerância a Falhas, Exclusão Mútua, Relógio Lógico

1 Introdução

Um banco de dados distribuído precisa manter os mesmos dados em várias máquinas ao mesmo tempo. Esse objetivo esconde três problemas clássicos de sistemas distribuídos: (i) uma escrita deve se tornar visível em *todas* as réplicas ou em *nenhuma*, mesmo que parte delas falhe no meio da operação; (ii) leituras atendidas por réplicas diferentes devem devolver o mesmo valor; e (iii) a queda de qualquer máquina, inclusive a que coordena as escritas, não pode interromper o serviço nem corromper os dados.

O sistema desenvolvido é um banco chave-valor replicado, acessado por um cliente de linha de comando com operações de escrita (`put`, `update`, `delete`) e de leitura (`get`, `exists`, `list`, `size`, `getAll`). As principais decisões metodológicas foram: replicação *síncrona* coordenada por um líder, priorizando consistência sobre disponibilidade; transações distribuídas com 2PC complementado por um protocolo de terminação; eleição de líder pelo algoritmo do valentão com salvaguarda contra estado defasado; e separação estrita entre a lógica distribuída e o transporte de rede, para permitir testes determinísticos de cada algoritmo.

2 Tecnologias Utilizadas

O sistema foi desenvolvido em Python 3, com gRPC para comunicação remota e Protocol Buffers para definição do contrato de serviço. O ambiente é isolado com `venv` e as dependências são fixadas em `requirements.txt`. Os algoritmos distribuídos (relógio lógico, exclusão mútua, eleição, 2PC, detector de falhas, terminação e quórum) foram implementados manualmente pelo grupo; a biblioteca gRPC é usada apenas como transporte.

3 Arquitetura do Sistema

O sistema é formado por N nós idênticos (os experimentos usam $N = 3$, mas a topologia é configurável por linha de comando) e por clientes. Todos os nós executam o mesmo programa e os papéis são definidos em tempo de execução:

- **Líder:** o nó vivo de maior identificador; é o único que coordena escritas, executando o 2PC sobre as réplicas vivas, e é a fonte de estado para nós que reingressam.
- **Réplicas (participantes):** armazenam os dados, votam no 2PC, atendem leituras e vigiam o líder por *heartbeat*; se ele morre, disparam uma eleição.
- **Cliente:** conhece a lista de nós; envia escritas ao líder (se contatar o nó errado recebe `NOT_LEADER` com o endereço correto e se redireciona) e lê de qualquer nó.

A topologia lógica é uma **malha completa**: todo nó se comunica diretamente com todos os outros. O *heartbeat* é todos-para-todos a cada 1 s, o 2PC flui do líder para as réplicas, a eleição contata os identificadores maiores e a terminação consulta qualquer par. Não há ponto único de roteamento; a queda de um nó não desconecta os demais. O sistema permite que cada nó execute em uma máquina da rede local; nos experimentos deste trabalho, a malha foi executada na própria máquina, com processos em portas distintas (50051..50053), mantendo comunicação gRPC/TCP real.

A organização do código reflete a separação entre lógica e transporte:

```
distributed-db/  
  proto/database.proto  contrato gRPC  
  distdb/  
    lamport.py          relógio de Lamport
```

store.py	armazem+WAL+locks
coordinator.py	2PC + terminacao
election.py	alg. do valentao
failure_detector.py	heartbeat
node.py	servidor gRPC (no)
client.py	cliente (failover)
scripts/	cluster, benchmark,
	estresse
tests/	unidade e integracao

Os módulos de `distdb/` (exceto `node.py` e `client.py`) não dependem de rede, o que permite exercitá-los em testes de unidade simulando falhas arbitrárias.

4 Comunicação RPC

Toda a comunicação, tanto cliente–nó quanto nó–nó, usa gRPC. O contrato em `database.proto` agrupa as mensagens em quatro famílias: API do cliente (`Put`, `Update`, `Delete`, `Get`, `Exists`, `ListKeys`, `Size`, `GetAll`); transação distribuída (`Prepare`, `Commit`, `Abort`, `QueryDecision`); coordenação (`Heartbeat`, `Election`, `Announce`, `WhoIsLeader`); e recuperação (`SyncState`). Toda mensagem carrega o relógio lógico do remetente, e todas as chamadas usam *timeouts* curtos, de modo que falhas de comunicação sejam detectadas rapidamente em vez de bloquearem o sistema.

5 Relógios Lógicos de Lamport

Cada processo mantém um contador inteiro seguindo as duas regras de Lamport [4]: incrementa antes de todo evento local ou envio de mensagem e, ao receber uma mensagem com carimbo t , atualiza para $\max(\text{local}, t) + 1$. Os carimbos estabelecem a relação *happened-before* entre eventos, datam as transações no registro durável e têm um papel central no controle de réplicas: o carimbo da última transação confirmada por um nó (`last_commit_ts`) é usado para comparar quão atualizado está o estado de cada réplica durante uma eleição (Seção 7).

6 Exclusão Mútua Distribuída

O controle de concorrência opera em dois níveis complementares. No **líder**, escritas concorrentes sobre a mesma chave são serializadas por um conjunto de *locks* locais (*striped locks*): requisições simultâneas de clientes diferentes são enfileiradas, enquanto chaves diferentes seguem em paralelo. É isso que sustenta alta concorrência sem desperdiçar trabalho com abortos (Seção 11). Em **cada participante**, durante a fase de votação do 2PC, o nó adquire um *lock* sobre a chave em seu armazém local; enquanto uma transação está preparada, qualquer outra que dispute a mesma chave recebe voto NÃO e é abortada. Esse segundo nível é a garantia *distribuída*: protege a consistência mesmo nas situações que escapam

à serialização do líder, como trocas de liderança no meio de uma transação.

7 Eleição de Líder

A eleição usa o algoritmo do valentão [2]: quando um nó percebe que o líder não responde, envia `ELECTION` a todos os nós de identificador maior; se ninguém responde, anuncia-se coordenador para todos (`Announce`); se algum nó maior responde, este assume a condução da eleição. O nó vivo de maior identificador sempre vence.

Uma salvaguarda complementa o algoritmo: um nó que ficou um período fora do ar (por exemplo, o antigo líder, justamente o de maior identificador) poderia vencer a eleição carregando estado defasado. Por isso o vencedor, *antes* de se anunciar, consulta os pares vivos (`SyncState`) e compara o `last_commit_ts` de cada um com o seu; se algum par tem estado mais recente, o vencedor o adota. Assim a liderança nunca regride o conteúdo do banco.

8 Transações Distribuídas (2PC)

Cada escrita é uma transação distribuída coordenada pelo líder em duas fases [3]. Antes de iniciar, o líder verifica o **quórum**: se o *cohort* (ele próprio mais as réplicas vivas) é menor que a maioria do agrupamento ($\lfloor N/2 \rfloor + 1$), a transação é recusada com `no quorum` (Seção 10).

Na fase de **votação**, o coordenador envia `PREPARE` com a operação a todos os participantes. Cada participante: adquire o *lock* da chave (Seção 6); valida a operação (`update` e `delete` exigem chave existente); torna a intenção durável no WAL; e vota SIM ou NÃO. Na fase de **decisão**, se todos votaram SIM o coordenador envia `COMMIT` e a escrita se torna visível atomicamente em todos; um único voto NÃO, ou a falha de um participante, gera `ABORT` em todos: ninguém aplica nada e os *locks* são liberados. O ponto fraco clássico do 2PC, o bloqueio de participantes quando o coordenador morre no meio do protocolo, é tratado pelo protocolo de terminação da Seção 9.6.

9 Tratamento de Falhas

9.1 Detecção por heartbeat

Cada nó envia `Heartbeat` aos demais a cada 1 s; após 3 s de silêncio (ou imediatamente após qualquer RPC falhar) o par é considerado suspeito. O detector alimenta duas decisões: o líder monta o *cohort* do 2PC apenas com réplicas vivas e as réplicas percebem a morte do líder.

9.2 Queda de réplica durante a escrita

Se uma réplica morre no meio de uma transação, o RPC falha e a transação é abortada (atomicidade preservada). O líder marca o nó como morto e *reexecuta* a escrita sobre

o *cohort* restante, de forma transparente ao cliente, desde que haja quórum.

9.3 Queda do líder

As réplicas detectam a ausência de *heartbeat* e disparam a eleição (Seção 7). O cliente redescobre o líder automaticamente: ao receber `NOT_LEADER` segue o redirecionamento e, diante de falhas de conexão, tenta os demais nós (*failover*).

9.4 Durabilidade e recuperação

Cada nó mantém um registro de escrita antecipada (WAL): os registros `PREPARE`, `COMMIT` e `ABORT` são forçados ao disco *antes* de qualquer alteração em memória [3]. Ao reiniciar, o nó reconstrói o estado a partir do *snapshot* mais recente e reaplica do WAL apenas as transações com `COMMIT` registrado; transações preparadas sem decisão são descartadas com segurança. Os *snapshots* compactam o log e são gerados nas transferências de estado (ressincronização de réplica ou adoção do estado mais recente na eleição).

9.5 Reingresso de réplica

Ao voltar, a réplica recupera o estado local pelo WAL e pede ao líder o estado completo (`SyncState`), ficando em dia inclusive com as escritas que perdeu enquanto esteve fora.

9.6 Transações em dúvida (terminação)

Se o coordenador morre entre o `PREPARE` e a decisão, o participante que votou `SIM` fica com a chave travada sem poder decidir sozinho. Implementamos a terminação cooperativa: transações preparadas há mais de 8 s sem decisão fazem o nó perguntar aos pares (`QueryDecision`) o que sabem, já que cada nó guarda de forma durável o desfecho das transações que decidiu. Se algum par registrou `COMMIT`, o nó confirma também; se ninguém viu decisão, aborta por presunção (*presumed abort*), o que é seguro: o coordenador só envia `COMMIT` após todos os votos `SIM`, logo, se nenhum par alcançável confirmou, nenhum cliente recebeu resposta de sucesso. Em ambos os casos o *lock* é liberado e o sistema se destrava sem intervenção manual.

10 Controle de Réplicas

O conjunto de réplicas é gerenciado por quatro mecanismos que atuam em conjunto: (i) o *cohort* de cada transação é montado dinamicamente com as réplicas vivas segundo o detector de falhas; (ii) escritas exigem **quórum majoritário** ($\lfloor N/2 \rfloor + 1$): numa partição de rede, apenas o lado com a maioria dos nós aceita escritas, e um líder isolado responde no *quorum* (as leituras do último estado confirmado continuam disponíveis), eliminando o

split-brain com líderes confirmando escritas divergentes; (iii) réplicas que reingressam são ressincronizadas por transferência de estado; e (iv) o vencedor de uma eleição adota o estado mais recente entre os pares vivos (`last_commit_ts`). O resultado é que o grupo de réplicas evolui sem perder escritas confirmadas nem ressuscitar estado antigo, numa escolha explícita de consistência em vez de disponibilidade.

11 Avaliação Experimental

A avaliação combinou três métodos. Os experimentos foram executados em um notebook com processador Intel Core i7-1260P (12ª geração, 12 núcleos/16 *threads*), 16 GB de RAM e SSD NVMe, sob Windows 11; os três nós e os clientes executaram na mesma máquina, comunicando-se por gRPC/TCP pela interface de *loopback*. A execução com cada nó em uma máquina física distinta da rede local está implementada, com topologia configurável via `-peers`, mas não foi testada nesta entrega: todos os experimentos relatados a seguir usaram uma única máquina.

Testes automatizados. Suíte de unidade e integração (diretório `tests/`) que exercita armazém, 2PC, eleição, detector, terminação e quórum sem rede. Os cenários incluem: queda de participante na fase de votação; coordenador que morre após confirmar em apenas um participante (resolvido pela terminação); recuperação por WAL que reaplica somente transações confirmadas; escrita recusada sem quórum; e o fluxo completo escrita $\rightarrow$ queda de réplica $\rightarrow$ queda do líder $\rightarrow$ eleição $\rightarrow$ ressincronização.

Injeção manual de falhas. Com o agrupamento no ar, derrubamos réplicas e o líder em momentos distintos e acompanhamos os registros. Na queda do líder (nó 3), o nó 2 detectou a ausência de *heartbeat*, venceu a eleição e assumiu em poucos segundos (a detecção é limitada pelo *timeout* de 3 s, mas costuma ser mais rápida, pois o RPC ao líder falha de imediato); a réplica restante ressincronizou o estado completo do novo líder em seguida. Também observamos a reexecução transparente de escritas após queda de réplica, a ressincronização no reingresso e a recusa de escritas (no *quorum*) quando restou apenas um nó.

Desempenho. O *benchmark* (200 operações de cada tipo) compara o agrupamento com um armazém local único, sem rede. O armazém local atingiu ≈ 472 mil escritas/s e $\approx 3,6$ milhões de leituras/s; o agrupamento sustentou 173,3 escritas/s (5,77 ms por operação) e 2,7 mil leituras/s (0,37 ms). A diferença de mais de três ordens de grandeza nas escritas (cerca de 2.700 vezes) é o preço da durabilidade e da replicação síncrona: cada escrita percorre duas rodadas de RPC (votação e decisão) e força o WAL ao disco (*fsync*) em cada um dos três participantes, enquanto o *baseline* apenas atualiza um dicionário em memória. As leituras distribuídas pagam somente uma RPC local, por isso ficam cerca de quinze vezes mais rápidas que as escritas. A Tabela 1 resume.

Alta concorrência. O teste de estresse disparou 8 clientes simultâneos com 100 operações cada (800 no total; 30% de leituras e 30% das escritas disputando 4 chaves “quentes”). O agrupamento sustentou 775,5 ops/s agregadas e concluiu **560 escritas confirmadas, sem nenhum aborto e nenhuma falha**: a serialização por chave no líder enfileira as escritas concorrentes na mesma chave em vez de deixá-las se abortarem no 2PC, ao custo de latência sob disputa (p50 de 12,2 ms, p95 de 21,8 ms e p99 de 29,5 ms nas escritas, contra p50 de 2,2 ms nas leituras). Ao final, o verificador leu o estado completo de cada nó e confirmou os três **idênticos**, com 594 chaves. Repetimos o teste derrubando o líder sob carga: a vazão cai durante a janela de reeleição (poucos segundos), os clientes reexecutam as escritas pendentes no novo líder e nenhuma operação se perde.

Table 1: Resultados de desempenho (medições do grupo).

Cenário	Escritas (ops/s)	Leituras (ops/s)
Máquina única (sem rede)	471 698	3 636 359
Distribuído, 3 nós (2PC)	173,3	2 688,7
Estresse: 8 clientes (carga mista)	542,8	232,7

12 Desafios e Problemas Encontrados

Os principais desafios foram: (i) reproduzir falhas de forma controlada para testar, resolvido separando a lógica distribuída do transporte gRPC, o que permite simular quedas em testes de unidade, complementado por injeção manual de falhas no agrupamento real; (ii) o caso do coordenador que morre no meio do 2PC, que exigiu estudar e implementar o protocolo de terminação e a semântica de *presumed abort*; (iii) impedir que um líder antigo reingressasse impondo estado defasado ao grupo, resolvido com a comparação de `last_commit_ts` na eleição; (iv) equilibrar exclusão mútua e vazão sob alta concorrência, que levou ao desenho em dois níveis da Seção 6; e (v) interpretar as medições com os três nós na mesma máquina: os processos disputam os mesmos núcleos e o mesmo disco, e a persistência (*fsync* do WAL em cada participante) domina a latência das escritas, o que exige cuidado ao extrapolar os números para um ambiente com máquinas separadas.

13 Conclusão e Melhorias Futuras

O sistema desenvolvido mantém réplicas consistentes sob falhas de réplicas, falha do coordenador e concorrência intensa, combinando 2PC com terminação cooperativa, eleição de líder com adoção de estado mais recente, quórum majoritário, WAL e transferência de estado. Os experimentos evidenciam o custo inerente da replicação síncrona nas escritas e a recuperação automática do

serviço em poucos segundos após falhas, sem perda de operações confirmadas.

Como melhorias futuras destacam-se: repetir a avaliação com cada nó em uma máquina física distinta da rede local (o suporte já está implementado, com topologia configurável via `-peers`, mas os experimentos desta entrega usaram uma única máquina); adotar um protocolo de consenso completo (Raft ou Paxos), com garantias formais também sob partições arbitrárias; WAL binário com soma de verificação; níveis de consistência configuráveis nas leituras; e particionamento do espaço de chaves (*sharding*) para escalar as escritas horizontalmente.

Declarations

Authors’ Contributions

Luís Augusto Starling Toledo foi responsável pela arquitetura do sistema, pela implementação do nó (servidor gRPC) e pela integração dos módulos.

Edson Pimenta de Almeida implementou o armazém chave-valor com WAL, *snapshots* e recuperação após falhas, além da organização do ambiente e do repositório.

Felipe Carvalho de Paula Silva foi responsável pelo contrato Protocol Buffers (mensagens de transação, eleição, terminação e recuperação), pela geração dos *stubs* e pelo cliente com redirecionamento e *failover*.

João Lucas de Melo Quintão desenvolveu a suíte de testes de unidade e integração e os roteiros de injeção de falhas.

Juan Pablo Ramos de Oliveira foi responsável pela coordenação das escritas via 2PC (Two-Phase Commit).

Luiz Gabriel Milione Assis implementou o detector de falhas por *heartbeat* e a eleição de líder, e preparou a execução do sistema em múltiplas máquinas (topologia configurável via `-peers`).

Túlio Gomes Braga conduziu a avaliação experimental (*benchmark* e teste de estresse), com análise dos registros e dos resultados.

References

- [1] Google. gRPC Documentation. Disponível em: <https://grpc.io/>
- [2] Garcia-Molina, H. (1982). Elections in a distributed computing system. *IEEE Transactions on Computers*, C-31(1):48–59.
- [3] Gray, J. e Reuter, A. (1993). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
- [4] Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565.
- [5] Tanenbaum, A. e van Steen, M. *Distributed Systems: Principles and Paradigms*. Prentice Hall.